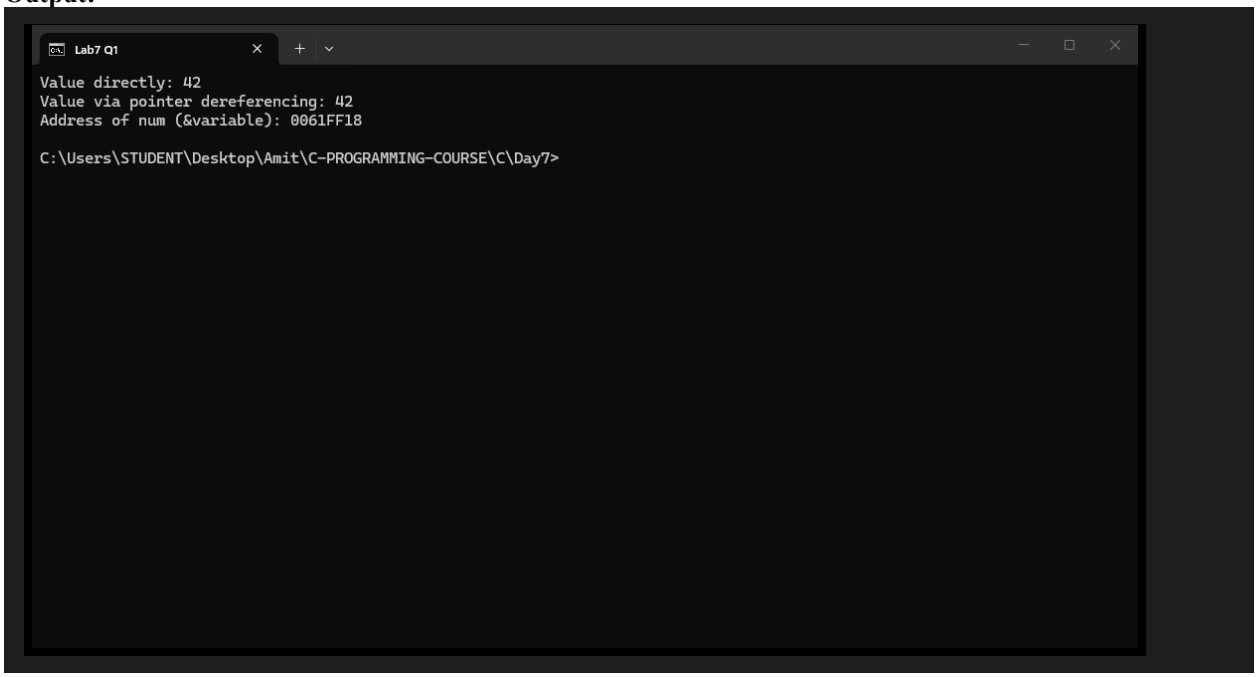
Amit_Lab7 - Day 7 Assignment

C Programming - Pointers

Question 1: Declare an integer, store its address in a pointer, and print the variable's value three ways - directly, via dereferencing the pointer, and via &variable.

```c
#include <stdio.h>

int main() {
    int num = 42;
    int *ptr;

    ptr = &num;

    printf("Value directly: %d\n", num);
    printf("Value via pointer dereferencing: %d\n", *ptr);
    printf("Address of num (&variable): %p\n", &num);

    return 0;
}
```

Output:

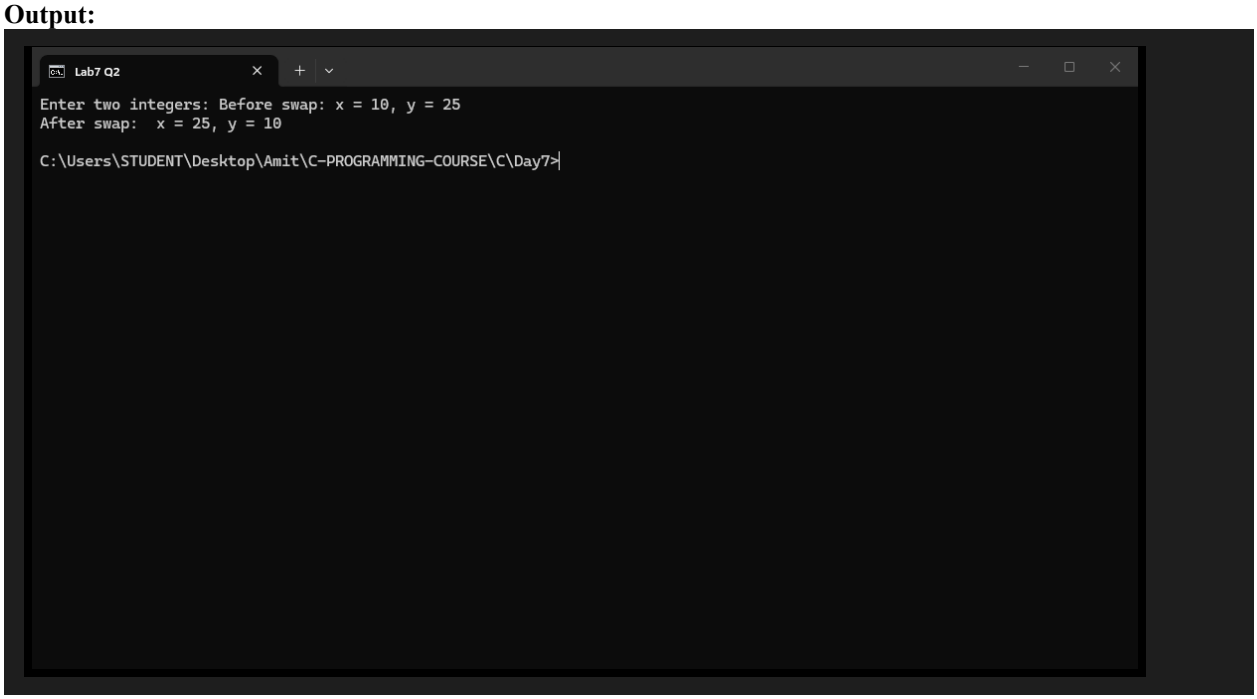


Question 2: Take two integers as input, and using only pointers swap their values. (Write a function swap(int *a, int *b)).

```c
#include <stdio.h>

void swap(int *a, int *b);

int main() {
    int x, y;

    printf("Enter two integers: ");
    scanf("%d %d", &x, &y);
```

```
    printf("Before swap: x = %d, y = %d\n", x, y);

    swap(&x, &y);

    printf("After swap:  x = %d, y = %d\n", x, y);

    return 0;
}

void swap(int *a, int *b) {
    int temp;

    temp = *a;
    *a = *b;
    *b = temp;
}
```

Output:

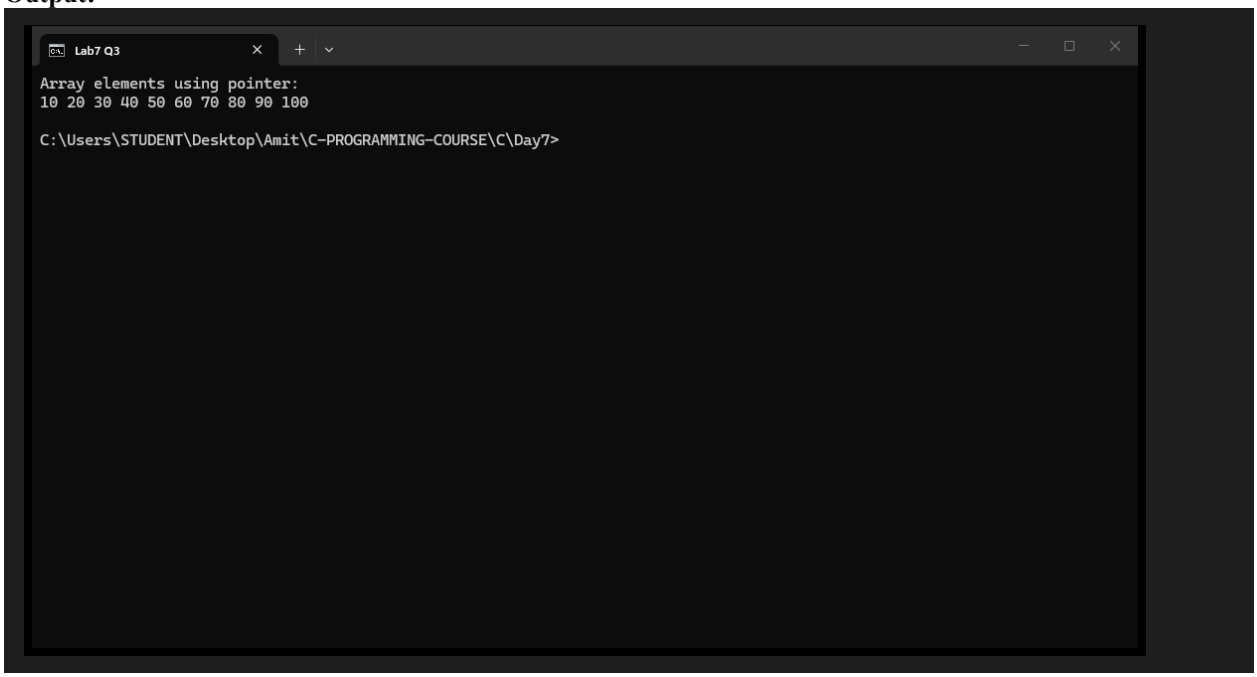


Question 3: Given an array of 10 integers, use a pointer (not indexing) to print every element.

```
#include <stdio.h>

int main() {
    int arr[10] = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};
    int *ptr;
    int i;

    ptr = arr;

    printf("Array elements using pointer:\n");
    for (i = 0; i < 10; i++) {
        printf("%d ", *ptr);
        ptr++;
    }
    printf("\n");

    return 0;
```

```
}
```

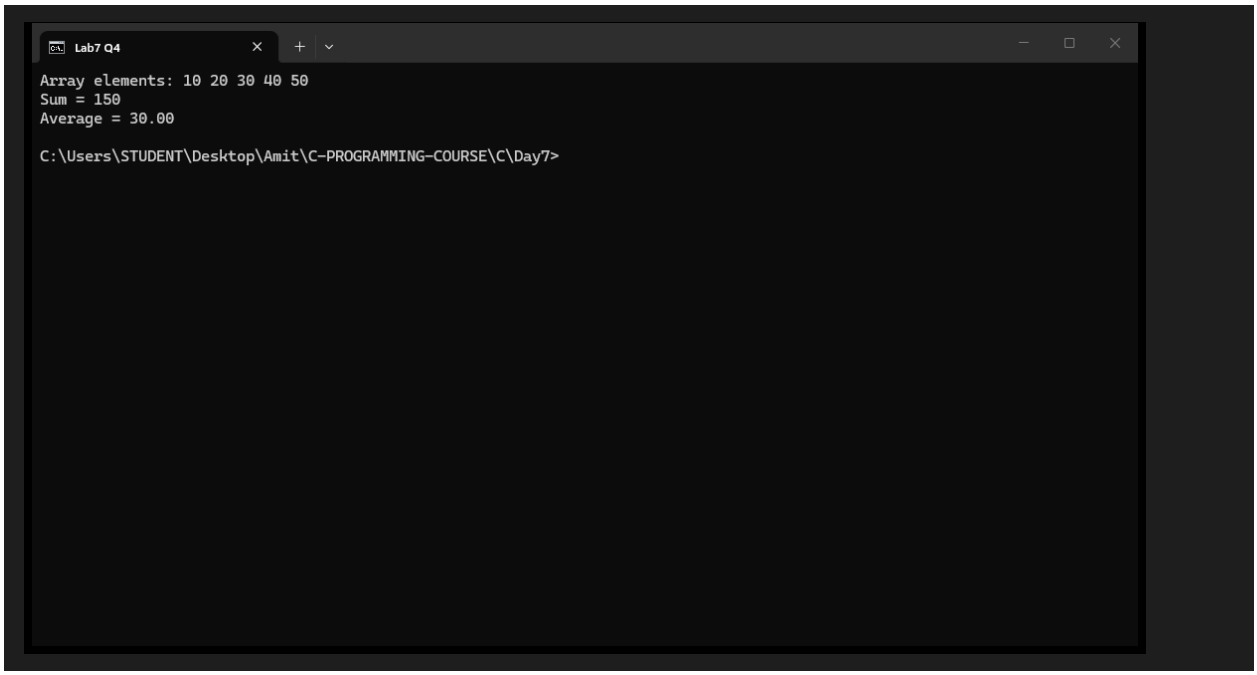
Output:



Question 4: Use a pointer to find the sum and average of all elements in an integer array.

```
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *ptr;
    int i;
    int sum = 0;
    float average;

    ptr = arr;

    for (i = 0; i < 5; i++) {
        sum = sum + *ptr;
        ptr++;
    }

    average = (float)sum / 5;

    printf("Array elements: ");
    ptr = arr;
    for (i = 0; i < 5; i++) {
        printf("%d ", *ptr);
        ptr++;
    }

    printf("\nSum = %d\n", sum);
    printf("Average = %.2f\n", average);

    return 0;
}
```

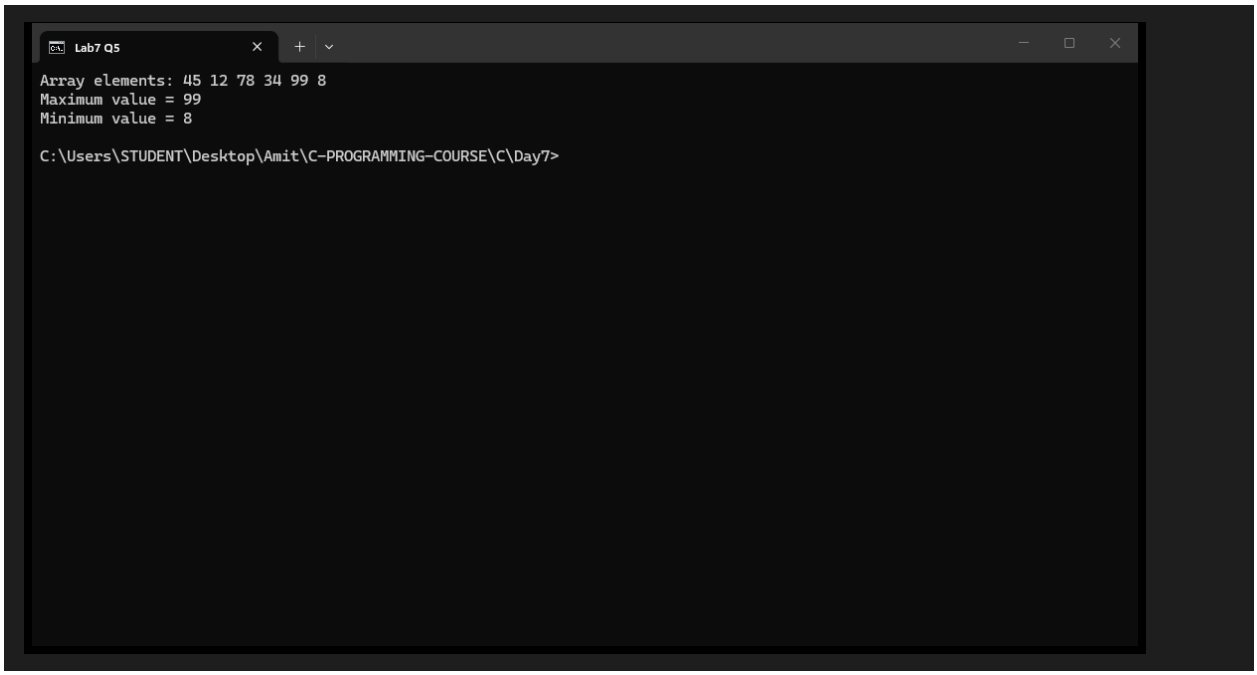
Output:



Question 5: Use a pointer to find the maximum and minimum value in an array, without using arr[i] anywhere.

```
#include <stdio.h>

int main() {
    int arr[6] = {45, 12, 78, 34, 99, 8};
    int *ptr;
    int i;
    int max, min;

    ptr = arr;

    max = *ptr;
    min = *ptr;

    for (i = 0; i < 6; i++) {
        if (*ptr > max) {
            max = *ptr;
        }
        if (*ptr < min) {
            min = *ptr;
        }
        ptr++;
    }

    printf("Array elements: ");
    ptr = arr;
    for (i = 0; i < 6; i++) {
        printf("%d ", *ptr);
        ptr++;
    }

    printf("\nMaximum value = %d\n", max);
    printf("Minimum value = %d\n", min);

    return 0;
}
```

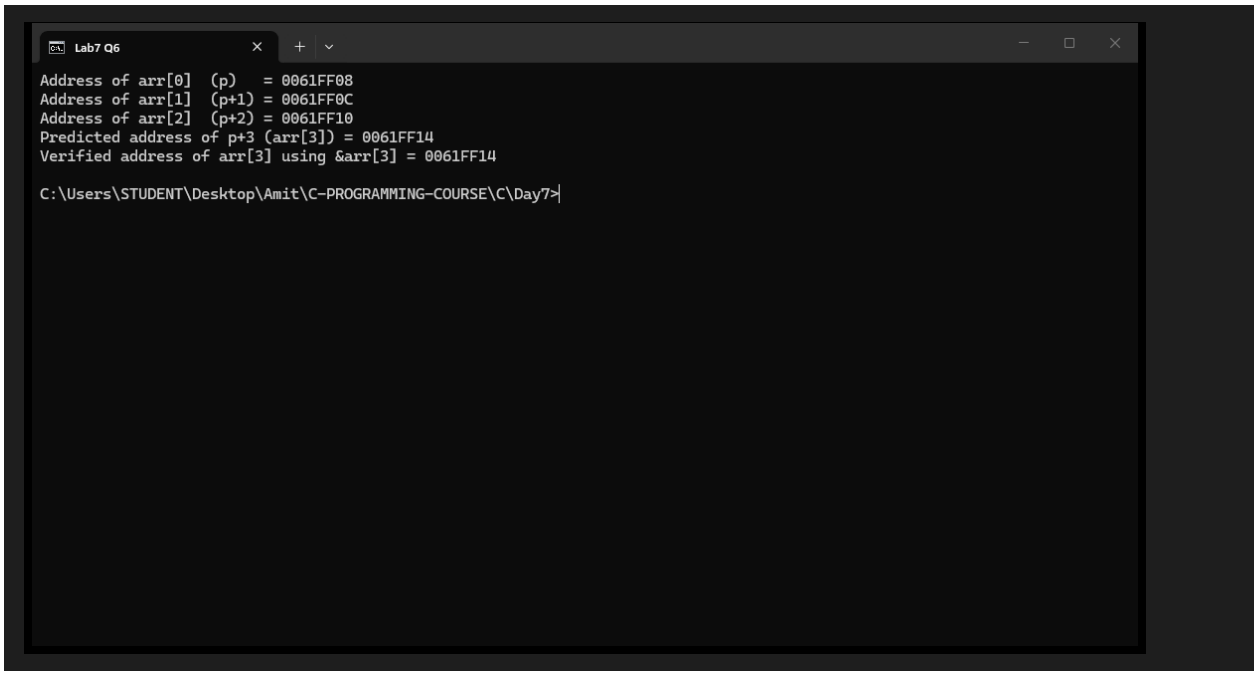
Output:



Question 6: Given a pointer p pointing to arr[0] of an int array, predict the address p + 3 will hold, then verify with printf("%p", ...).

```c
#include <stdio.h>

int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p;

    p = &arr[0];

    printf("Address of arr[0]  (p)   = %p\n", (void*)p);
    printf("Address of arr[1]  (p+1) = %p\n", (void*)(p + 1));
    printf("Address of arr[2]  (p+2) = %p\n", (void*)(p + 2));
    printf("Predicted address of p+3 (arr[3]) = %p\n", (void*)(p + 3));
    printf("Verified address of arr[3] using &arr[3] = %p\n", (void*)&arr[3]);

    return 0;
}
```

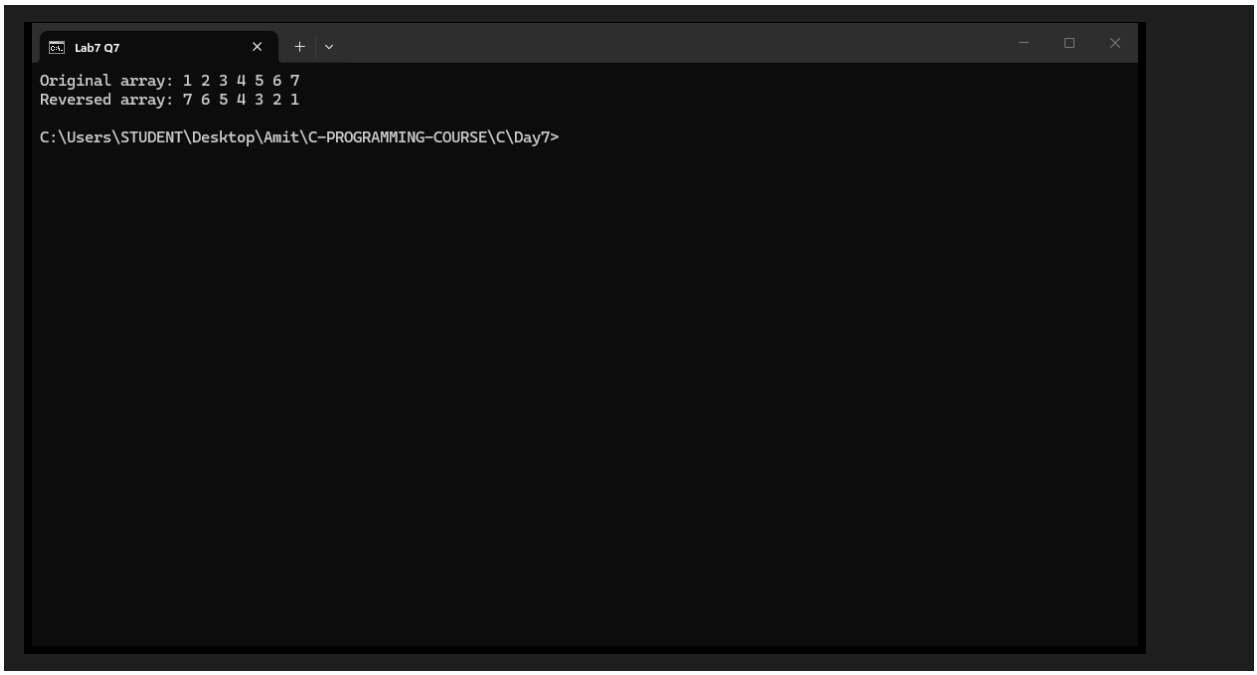
Output:



Question 7: Reverse an array in place using two pointers - one starting at the first element, one at the last, swapping and moving inward until they meet.

```c
#include <stdio.h>

int main() {
    int arr[7] = {1, 2, 3, 4, 5, 6, 7};
    int *left, *right;
    int temp;
    int i;

    printf("Original array: ");
    for (i = 0; i < 7; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    left = arr;
    right = arr + 6;

    while (left < right) {
        temp = *left;
        *left = *right;
        *right = temp;
        left++;
        right--;
    }

    printf("Reversed array: ");
    for (i = 0; i < 7; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

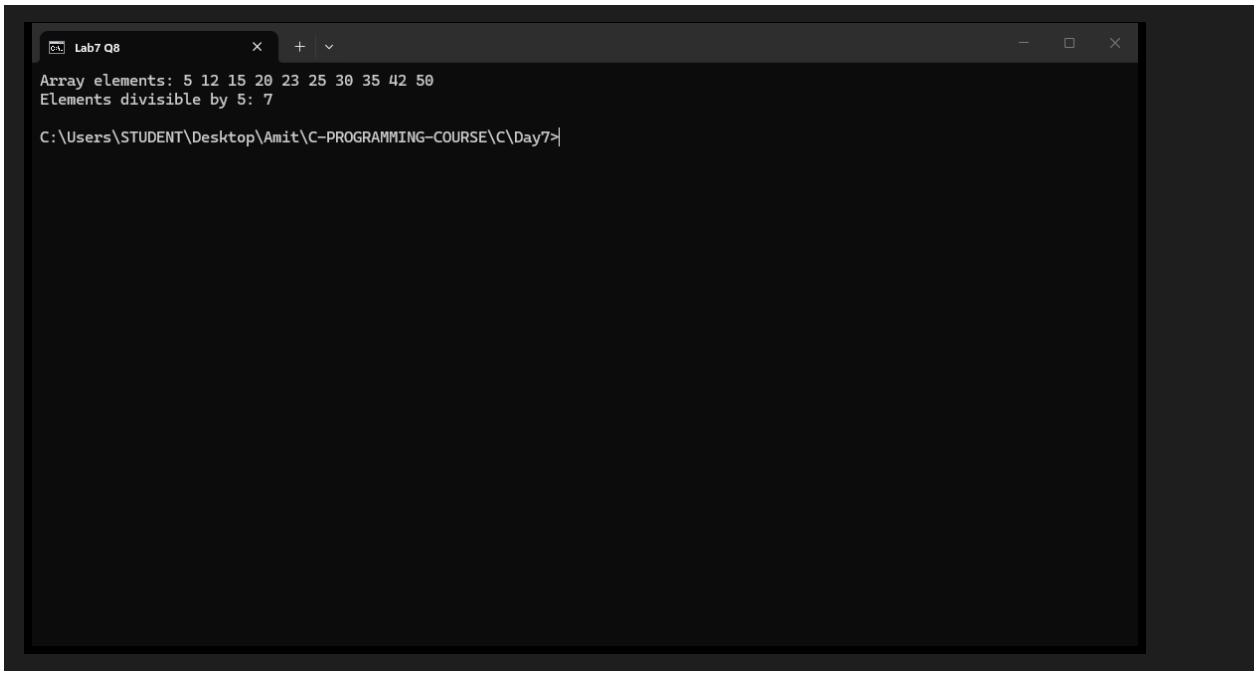
Output:



Question 8: Write a program using pointer arithmetic to count how many elements in an array are divisible by 5.

```
#include <stdio.h>

int main() {
    int arr[10] = {5, 12, 15, 20, 23, 25, 30, 35, 42, 50};
    int *ptr;
    int count = 0;
    int i;

    ptr = arr;

    for (i = 0; i < 10; i++) {
        if (*ptr % 5 == 0) {
            count++;
        }
        ptr++;
    }

    printf("Array elements: ");
    ptr = arr;
    for (i = 0; i < 10; i++) {
        printf("%d ", *ptr);
        ptr++;
    }

    printf("\nElements divisible by 5: %d\n", count);

    return 0;
}
```

Output:



Explain

1. What is a pointer and what does the address-of operator (&) do?
A pointer is a variable that stores the memory address of another variable instead of a value. The address-of operator (&) gives the memory address of a variable, for example &num returns the address where num is stored. In Question 1 the pointer ptr is initialized as ptr = # and then *ptr reads the value stored at that address. The value 42 is printed directly, through dereferencing, and from the address in the pointer.
2. How do you declare a pointer and access the value it points to (dereferencing)?
A pointer is declared by putting a * before its name, using the type of the data it points to, for example int *ptr;. The value at the address is accessed with the dereference operator *ptr. In Question 1, *ptr gives 42 which is the same value as num. The * in the declaration means "pointer to", while * in an expression means "value at the address", which confuses many beginners.
3. Why does the swap function need pointer parameters? Why does call by value fail?
In C, function parameters receive a copy of the argument (call by value). If swap took int a and int b, it would swap only the copies and the original x and y in main() would stay unchanged. Passing &x and &y gives swap the actual addresses, and *a and *b access the original variables, so the swap works in main(). This is the standard reason pointers are used to modify caller variables from inside a function.
4. What is pointer arithmetic? How much is p + 3 added in Question 6?
Pointer arithmetic works in units of the pointed-to type, not bytes. For an int array, p + 1 points to the next int element. Since an int is 4 bytes on this system, p + 3 advances the address by 3 * 4 = 12 bytes. The program predicts the address of arr[3] as p + 3 and verifies it with &arr[3], and both print the same address (0061FF14 in the sample run).
5. Explain how the two-pointer reverse loop in Question 7 works.
Two pointers are used: left starts at arr[0] and right starts at arr[6] (the last element). The loop runs while left < right. Each iteration swaps *left and *right, then moves left forward with left++ and right backward with right--. When the two pointers meet in the middle, the whole array is reversed in place without using any extra array. The original 1 2 3 4 5 6 7 becomes 7 6 5 4 3 2 1.